SALES ANALYTICS

StyleSense is a growing clothing retail company that sells Men's, Women's, and Kids' clothing across Physical stores and Online channels. Like many retail businesses today, StyleSense faces growing competition from large e-commerce platforms and needs to make smarter, data-driven decisions to stay profitable.

Business Problem

Despite consistent sales, StyleSense's :

- Profit margins are unclear – cost of goods, payroll, rent, and marketing all eat into revenue.
- Seasonality is not being planned for – winter and summer products go out of stock or sit unsold.
- Promotions and discounts are used freely – but no one knows if they're actually generating profit.
- Customer segments are not being leveraged – VIP customers may be leaving due to lack of retention programs.
- Online vs physical performance is unknown – marketing spend may be mis-allocated.

Project Objective

This project analyzes 18 months (Jan 2024 – Jun 2025) of sales, customer, inventory, and expense data to:

- Understand what has been selling and who is buying (Descriptive).
- Diagnose why performance is the way it is (Diagnostic).
- Predict future revenue trends (Predictive).
- Recommend clear actions management can take to improve ROI (Prescriptive).

Data Understanding

- clothing_sales.csv(10,000 rows)-every transaction: product, channel, price, discount, profit.
- clothing_customers.csv(200 rows)-customer profiles: demographics, segments, purchase history.
- clothing_expenses.csv(1,300 rows)-business costs by category (rent, payroll, COGS, marketing).
- clothing_inventory.csv(13,600 rows)-daily stock levels per product with reorder triggers.
- clothing_products.csv(25 rows)-product master list with base price and cost.

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from sklearn.linear_model import LinearRegression
```

```python
from sklearn.metrics import r2_score, mean_absolute_error
import warnings

warnings.filterwarnings('ignore')
```

In [2]:

```python
# Chart styling
sns.set_theme(style="whitegrid")
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 10

BLUE   = '#3498DB'
GREEN  = '#27AE60'
RED    = '#E74C3C'
ORANGE = '#E67E22'
PURPLE = '#8E44AD'
GOLD   = '#F39C12'
TEAL   = '#1ABC9C'
GREY   = '#95A5A6'
```

In [3]:

```python
# Load all five datasets
sales     = pd.read_csv('clothing_sales.csv')
customers = pd.read_csv('clothing_customers.csv')
expenses  = pd.read_csv('clothing_expenses.csv')
inventory = pd.read_csv('clothing_inventory.csv')
products  = pd.read_csv('clothing_products.csv')
```

In [4]:

```python
print(f'  Sales rows:      {len(sales):,}')
print(f'  Customer rows:   {len(customers):,}')
print(f'  Expense rows:    {len(expenses):,}')
print(f'  Inventory rows:  {len(inventory):,}')
print(f'  Product rows:    {len(products):,}')
```

```
Sales rows:      10,039
Customer rows:   200
Expense rows:    1,322
Inventory rows:  13,675
Product rows:    25
```

In [5]:

```python
#Examine sales data structure
#Looking at the last few rows gives me a sense of: what columns exist,data types (dat
#and the most recent transactions (end of June 2025)
sales.tail()
```

Out[5]:

	sale_id	date	store_channel	payment_method	product_id	product_name	category
10034	S01581	2024-03-22	Physical	Credit Card	P010	Women's Blouse	Women
10035	S05467	2024-10-29	Physical	Debit Card	P021	Unisex Socks (3-pack)	All
10036	S06941	2025-01-11	Online	PayPal	P006	Men's Puffer Jacket	Men
10037	S06299	2024-12-17	Online	Debit Card	P021	Unisex Socks (3-pack)	All
10038	S00246	2024-01-07	Online	PayPal	P020	Unisex Cap	All

Data Cleaning_ Sales_csv

In [6]:

```
#Check for missing values
sales.isnull().sum()
```

Out[6]:

```
sale_id              0
date                 0
store_channel        0
payment_method       5
product_id           0
product_name         0
category             0
sub_category         0
quantity_sold        0
unit_price           0
discount_pct         0
discount_amount      0
total_sale           0
cost_total           0
gross_profit         0
customer_id        101
is_promotion         0
is_returned          0
dtype: int64
```

There are :

- 5 missing payment_methods
- 101 missing customer_ids

In [7]:

```
#check for nulls in sale_id column
sales.sale_id.isnull().sum()
```

Out[7]:

```
np.int64(0)
```

In [8]:

```
#check for duplicates in sale_id column
sales['sale_id'].duplicated().sum()
```

Out[8]: np.int64(50)

- There are 50 duplicated ids in sales_id column.

In [9]:

```
# See the duplicate rows
dupes = sales[sales['sale_id'].duplicated(keep=False)].sort_values('sale_id')
print(dupes[['sale_id', 'date', 'customer_id', 'product_id', 'quantity_sold', 'total_
```

	sale_id	date	customer_id	product_id	quantity_sold	total_sale
32	S00033	2024-01-02	C150	P020	1	11.99
9993	S00033	2024-01-02	C150	P020	1	11.99
145	S00146	2024-01-04	C047	P018	1	38.24
10020	S00146	2024-01-04	C047	P018	1	38.24
10038	S00246	2024-01-07	C111	P020	3	44.97
245	S00246	2024-01-07	C111	P020	3	44.97
248	S00249	2024-01-07	C028	P023	1	23.74
9989	S00249	2024-01-07	C028	P023	1	23.74
10031	S00273	2024-01-07	C200	P013	3	152.97
272	S00273	2024-01-07	C200	P013	3	152.97
816	S00817	2024-02-10	C179	P005	3	170.97
10004	S00817	2024-02-10	C179	P005	3	170.97
1033	S01034	2024-02-17	C045	P011	1	39.99
10010	S01034	2024-02-17	C045	P011	1	39.99
1157	S01158	2024-02-25	C141	P010	2	69.98
10011	S01158	2024-02-25	C141	P010	2	69.98
1472	S01473	2024-03-17	C184	P018	1	44.99
10000	S01473	2024-03-17	C184	P018	1	44.99
1580	S01581	2024-03-22	C124	P010	3	94.47
10034	S01581	2024-03-22	C124	P010	3	94.47

- The duplicates appear to be exact duplicates,suggesting a system glitch during data export. So i remove duplicates and I keep='first' to preserve one copy of each transaction. This reduces my dataset from 10,039 to 9,989 rows - a 50-row reduction.

In [10]:

```
# drop duplicates
sales = sales.drop_duplicates(subset='sale_id', keep='first')
print(sales.shape)
```

```
(9989, 18)
```

In [11]:

```
#check customer_id column
sales.customer_id
```

Out[11]:
```
0       C032
1       C151
2       C010
3       C017
4       C089
        ...
9984    C052
9985    C105
9986    C109
9987    C028
9988    C143
Name: customer_id, Length: 9989, dtype: str
```

In [12]:
```
# check for duplicates
sales.customer_id.duplicated().sum()
```

Out[12]:
```
np.int64(9788)
```

- The duplicated values in customer_id are likely to be because a customer can make purchases in different days, making them a repeat client.

In [13]:
```
# check column names
sales.columns
```

Out[13]:
```
Index(['sale_id', 'date', 'store_channel', 'payment_method', 'product_id',
       'product_name', 'category', 'sub_category', 'quantity_sold',
       'unit_price', 'discount_pct', 'discount_amount', 'total_sale',
       'cost_total', 'gross_profit', 'customer_id', 'is_promotion',
       'is_returned'],
      dtype='str')
```

In [14]:
```
# check for column data types
sales.dtypes
```

Out[14]:
```
sale_id                str
date                   str
store_channel          str
payment_method         str
product_id             str
product_name           str
category               str
sub_category           str
quantity_sold        int64
unit_price         float64
discount_pct         int64
discount_amount    float64
total_sale         float64
cost_total         float64
gross_profit       float64
customer_id            str
is_promotion          bool
is_returned           bool
dtype: object
```

In [15]:

```
# convert date column ino data data type
sales.date= pd.to_datetime(sales['date'])
```

In [16]:

```
# convert all string columns to object data type
str_cols = sales.select_dtypes(include='string').columns
sales[str_cols] = sales[str_cols].astype(object)
sales.dtypes
```

Out[16]:

```
sale_id                    object
date               datetime64[us]
store_channel              object
payment_method             object
product_id                 object
product_name               object
category                   object
sub_category               object
quantity_sold               int64
unit_price                float64
discount_pct                int64
discount_amount           float64
total_sale                float64
cost_total                float64
gross_profit              float64
customer_id                object
is_promotion                 bool
is_returned                  bool
dtype: object
```

In [17]:

```
#statistical view of date column
sales.date.describe()
```

Out[17]:

```
count                           9989
mean     2024-09-28 10:34:43.812193
min             2024-01-01 00:00:00
25%             2024-05-15 00:00:00
50%             2024-09-29 00:00:00
75%             2025-02-11 00:00:00
max             2025-06-30 00:00:00
Name: date, dtype: object
```

In [18]:

```
sales['date'].value_counts()
```

Out[18]:
```
date
2024-08-22    50
2024-01-06    49
2025-01-01    49
2025-06-03    49
2025-06-04    49
              ..
2025-04-09     8
2025-05-01     8
2025-06-12     8
2025-06-20     8
2025-06-23     8
Name: count, Length: 547, dtype: int64
```

In [19]:
```
sales['date'].duplicated().sum()
```

Out[19]:
```
np.int64(9442)
```

In [20]:
```
# Check sales count per day to spot any suspicious gaps
sales.groupby('date').size().sort_values()
```

Out[20]:
```
date
2025-06-12     8
2024-02-06     8
2025-02-20     8
2024-01-08     8
2025-01-15     8
              ..
2025-01-01    49
2024-01-06    49
2025-06-03    49
2025-06-04    49
2024-08-22    50
Length: 547, dtype: int64
```

In [21]:
```
# Check if any dates fall on unexpected days (e.g. if store is closed on certain days
sales['date'].dt.dayofweek.value_counts()
```

Out[21]:
```
date
5    1871
6    1833
4    1295
3    1266
0    1244
1    1244
2    1236
Name: count, dtype: int64
```

- All days have sales, but weekends (5,6) have highest volume. This makes sense for retail and validates data quality.

In [22]:
```
sales[sales['date'] == '']
```

Out[22]:

sale_id	date	store_channel	payment_method	product_id	product_name	category	sub_cat

In [23]:

```
#Analyze discount patterns
sales['discount_pct'].value_counts()
```

Out[23]:

```
discount_pct
0     4489
5     2189
10    1829
15    1127
20     355
Name: count, dtype: int64
```

- Discounts are in 5% increments (0,5,10,15,20%).
- This suggests standardized promotions rather than arbitrary discounts.
- 0% discounts (4,489 transactions) might be full-price sales or non-promotional items.

Validate and correct financial calculations

In [24]:

```
# Check if discount was calculated per unit instead of total
sales['discount_check_unit'] = (sales['unit_price'] * sales['discount_pct'] / 100).ro
sales[sales['discount_check_unit'] != sales['discount_amount']][['unit_price', 'quant
```

Out[24]:

unit_price	quantity_sold	discount_pct	discount_amount	discount_check_unit

In [25]:

```
sales['discount_pct'].describe()
```

Out[25]:

```
count    9989.000000
mean        5.329863
std         5.914879
min         0.000000
25%         0.000000
50%         5.000000
75%        10.000000
max        20.000000
Name: discount_pct, dtype: float64
```

Recalculate financial metrics

In [26]:

```
sales['discount_amount'] = (sales['unit_price'] * sales['quantity_sold'] * sales['dis
```

In [27]:

```
sales['total_sale'] = (sales['unit_price'] * sales['quantity_sold'] - sales['discount
sales['gross_profit'] = (sales['total_sale'] - sales['cost_total']).round(2)
```

In [28]:

```
# discount_amount should be unit_price * quantity * discount_pct/100
sales['discount_check'] = (sales['unit_price'] * sales['quantity_sold'] * sales['disc
sales[sales['discount_check'] != sales['discount_amount']]

# gross_profit should be total_sale - cost_total
sales['profit_check'] = (sales['total_sale'] - sales['cost_total']).round(2)
sales[sales['profit_check'] != sales['gross_profit']]
```

Out[28]:

sale_id	date	store_channel	payment_method	product_id	product_name	category	sub_cat

0 rows × 21 columns

In [29]:

```
# drop the temporary columns
sales.drop(columns=['discount_check', 'discount_check_unit', 'profit_check'], inplace
```

In [30]:

```
#check column payment_method
sales.payment_method
```

Out[30]:

```
0        Debit Card
1        Debit Card
2       Credit Card
3              Cash
4       Credit Card
           ...
9984     Debit Card
9985    Credit Card
9986    Credit Card
9987    Credit Card
9988     Debit Card
Name: payment_method, Length: 9989, dtype: object
```

In [31]:

```
#check for nique values in this column
sales.payment_method.unique()
```

Out[31]:

```
array(['Debit Card', 'Credit Card', 'Cash', 'Mobile Pay', 'PayPal', nan],
      dtype=object)
```

In [32]:

```
#Handle missing payment methods
sales.loc[sales['payment_method'].isna(), 'payment_method'] = 'Unknown'
```

In [33]:

```
#Handle missing customer_ids
sales.loc[sales['customer_id'].isna(), 'customer_id'] = 'Guest'
```

There are 101 missing values in customer_id column and could mean:

- Walk-in customers not in loyalty program.
- Online guest checkouts.

- Data entry errors.

So filling with 'Guest' category lets me analyze these separately without losing sales data.

In [34]:
```
# Verify all nulls handled
sales.isnull().sum()
```

Out[34]:
```
sale_id            0
date               0
store_channel      0
payment_method     0
product_id         0
product_name       0
category           0
sub_category       0
quantity_sold      0
unit_price         0
discount_pct       0
discount_amount    0
total_sale         0
cost_total         0
gross_profit       0
customer_id        0
is_promotion       0
is_returned        0
dtype: int64
```

In [35]:
```
#Save cleaned data
sales.to_csv('sales_clean_data.csv', index=False)
```

Data Cleaning_ customers_csv

In [36]:
```
customers.head()
```

Out[36]:

	customer_id	name	email	age	gender	city	state
0	C001	Sandra Lewis	sandra.lewis1@yahoo.com	40	F	Nashville	TN
1	C002	Kimberly Martinez	kimberly.martinez98@outlook.com	33	F	Charlotte	NC
2	C003	Susan Thompson	susan.thompson78@outlook.com	42	F	Atlanta	GA
3	C004	Anna Davis	anna.davis11@icloud.com	53	F	Minneapolis	MN
4	C005	Pamela Gonzalez	pamela.gonzalez6@yahoo.com	32	F	Phoenix	AZ

In [37]:
```
customers.tail()
```